Informe final – Optimización de cartera P2P “PrestaMatch”

Equipo: Barbero – Belmonte – Galván · Inteligencia Artificial · UCSE 2026

1. Cómo modelamos el problema

El objetivo no es clasificar bien, sino maximizar la ganancia neta de la campaña de enero 2026 sin superar el presupuesto de USD 2.000.000. Por eso dividimos el problema en dos partes: (1) un modelo de clasificación que estima, para cada solicitud, la probabilidad p de que el préstamo se pague; y (2) una regla de decisión económica que, a partir de p, el monto y la tasa, elige qué préstamos aprobar dentro del presupuesto.

2. Datos y decisiones sobre la entrada

Train tiene 32.416 solicitudes (enero a diciembre 2025) y test 3.061 solicitudes de enero 2026. El comportamiento bancario tiene hasta 24 registros mensuales por usuario (2024-01 a 2025-12), que hay que resumir en una fila por solicitud.

- Data leakage temporal. El historial bancario llega hasta 2025-12 para todos los usuarios, incluso para quienes pidieron el préstamo a principios de 2025. Usar registros posteriores a una solicitud determinada sería usar información del futuro (por ejemplo, atrasos causados por el propio default). Por eso solo usamos los meses estrictamente anteriores a la Fecha_Solicitud.
- Ventana de 12 meses. Las solicitudes de enero 2025 solo tienen 12 meses de historia previa, y las de test tienen 24. Para que las features se calculen igual en train y en test, usamos los 12 meses anteriores a cada solicitud. Con una ventana de 24 meses, las variables de test (por ejemplo, el atraso máximo) se calcularían sobre períodos más largos que las de train. Aún así quisimos corroborar el desempeño de esta forma, y al probarlo no mejoraba.
- Agregación por solicitud, no por usuario. Los usuarios pueden tener más de un préstamo en train, y cada préstamo tiene su propio "pasado". Las features se calculan con un groupby por ID_Solicitud, a partir de la historia de ese usuario. Ningún cálculo usa información de otras filas ni del target.
- Features construidas: saldo promedio, mínimo y desvío; atraso máximo y promedio; cantidad de meses con más de 30 días de atraso; meses informados; saldo y atraso máximo de los últimos 3 meses; tendencia del saldo (últimos 3 meses / promedio); monto/ingreso y saldo/ingreso. A esto se suman Monto, Tasa, Ingreso y Motivo (categórica). Descartamos los IDs y la fecha, porque test tiene un mes que el modelo nunca vio, intentando también evitar overfitting.

3. Modelo y validación

Usamos HistGradientBoostingClassifier (ensamble de árboles por boosting, que maneja variables categóricas y valores faltantes sin preprocesamiento) con regularización: max_depth=4, min_samples_leaf=50, learning_rate=0.05, max_iter=400 y random_state=42. Sobre una partición estratificada 80/20 obtuvo un AUC de 0,84.

Como la métrica es la ganancia, validamos simulando la regla de decisión completa sobre datos donde conocemos el resultado. Para que la validación se pareciera a test, entrenamos con todos los préstamos (el modelo aprende de todos), pero medimos la ganancia solo sobre préstamos de hasta USD 11.000, y promediamos 5 particiones con distintas semillas para no depender de la suerte de una sola. Sin este filtro, la validación premiaba estrategias que elegían unos pocos préstamos muy grandes, una situación que en test no puede ocurrir.

4. Regla de decisión

Para cada solicitud calculamos la esperanza matemática de la ganancia:

E = p · Monto · Tasa − (1 − p) · Monto
Aprobar solo si E > 0 equivale a exigir p > 1/(1 + Tasa). Es un umbral de probabilidad que se ajusta solo según la tasa de cada préstamo. Luego ordenamos las solicitudes por rentabilidad = E / Monto (ganancia esperada por dólar prestado). Por último, recorremos la lista y aprobamos cada préstamo si E > 0 y si todavía entra en el presupuesto restante. Una primera versión usaba un acumulado (cumsum) que rechazaba también a todos los préstamos posteriores al primero que no entraba, aunque hubiera alguno más chico que sí entrara.

5. Resultados y otros enfoques probados

Variante (validación: préstamos ≤ 11.000, promedio de 5 particiones)	Ganancia (USD)	Comentario
Gradient Boosting regularizado + orden por rentabilidad	530.909	Modelo final (entrega 3)
Gradient Boosting sin regularizar + orden por rentabilidad	530.571	Empate: no había sobreajuste
Regresión logística + orden por rentabilidad	521.936	Menor ganancia; elegida como 2.ª final (entrega 4)
Random Forest + orden por rentabilidad	516.897	Aún menor ganancia.
Gradient Boosting + orden por ganancia esperada en USD	481.498	Concentra en préstamos grandes, por lo que fallar en uno equivale a grandes pérdidas
Gradient Boosting + orden por probabilidad de pago	467.625	Ignora la tasa

- Lo que más impacta es la regla de decisión, no el modelo: ordenar mal cuesta entre un 9% y un 12%, mientras que cambiar de modelo mueve entre el 1% y el 3%.
- Umbrales fijos de probabilidad. Un umbral de 60% no cambia nada, porque la condición E > 0 ya exige alrededor de 79% con los datos de prueba. Umbrales mayores al 94% empeoraban la ganancia, porque reemplazan préstamos de tasa alta por otros más seguros pero menos rentables.
- No mejoraron: features bancarias adicionales (mediana de atraso, proporción de meses con atraso, valores del último mes), ventana de 24 meses (empate) o de 6 meses (bajó un poco), calibración de probabilidades y promediar gradient boosting con regresión logística.
- Como entregas finales elegimos la 3 y la 4: dos modelos distintos, porque el puntaje final toma la mejor de las dos y así diversificamos.

6. Limitaciones

Train y test difieren en el tamaño de los préstamos: el modelo aprendió sobre todo de préstamos grandes, aunque captura que los chicos se pagan más. Nuestra validación separa por fila y no por usuario, por lo que puede sobreestimar levemente el rendimiento: al separar por usuario, las conclusiones no cambiaron. Usamos herramientas de IA como asistente para entender documentación, depurar errores y discutir alternativas.